Soft Begging: Modular and Efficient Shielding of LLMs against Prompt Injection and Jailbreaking based on Prompt Tuning

Simon Ostermann[1,2], **Kevin Baum**[1,2], **Christoph Endres**[3], **Julia Masloh**[3], **Patrick Schramowski**[1,2]
[1]Deutsches Forschungszentrum für Künstliche Intelligenz (DFKI)
[2]Centre for European Research in Trusted AI (CERTAIN)
[3]sequire technology GmbH
firstname.lastname@{dfki|sequire}.de

arXiv:2407.03391v1 [cs.CR] 3 Jul 2024

Abstract

Prompt injection (both direct and indirect) and jailbreaking are now recognized as significant issues for large language models (LLMs), particularly due to their potential for harm in application-integrated contexts. This extended abstract explores a novel approach to protecting LLMs from such attacks, termed "soft begging." This method involves training soft prompts to counteract the effects of corrupted prompts on the LLM's output. We provide an overview of prompt injections and jailbreaking, introduce the theoretical basis of the "soft begging" technique, and discuss an evaluation of its effectiveness.

1 Background: Attacking LLMs

Current LLMs lack adversarial robustness (Carlini et al., 2021). This leads to new phenomena observed in the attack surface of LLMs that are relevant to safety and security. Prominent examples include: jailbreaking, direct prompt injections and indirect prompt injections.

In jailbreaking, a target LLM is manipulated to circumvent content moderation systems and subvert safety rules (Zou et al., 2023). Prompt injections describe attacks where input is inserted into an application and processed by the LLM later downstream - with indirect prompt injections denoting those cases where the attacker is not the user prompting the LLM (Greshake et al., 2023). As can be seen, direct prompt injection (Perez and Ribeiro, 2022) is similar to jailbreaking: Both hide specific instructions inside a prompt that alters or manipulates the behavior of the system in a manner not intended by the LLM provider. This adversarial instruction is typically conveyed through text which is sometimes visually concealed (using tactics such as white or extremely small font sizes), encoded (Liu et al., 2024b) or obfuscated by an API (for example, a command that is appended to each input without the end user's awareness).

Especially indirect prompt injection attacks, where adversaries remotely affect the target system, exploit the fact that there is no explicit separation between code and data in current LLMs and, as a consequence, between valid system instructions and invalid, potentially adversarial, instructions (Zverev et al., 2024).

Dangers of prompt injections include data leakage, i.e., the disclosure of sensitive information, and system manipulation, i.e., altering the behaviour of the system based on the injected prompt. In the most extreme case, the adversarial user can hijack the LLM (Qiang et al., 2024), which is even more problematic if the LLM has great autonomy and is, for example, given access to plugins or third-party data.

As the goal of the examples mentioned above is often to elicit harmful behavior of an LLM, it is important to note that there are also benign and neutral usages. Benign injections may guide the model to generate more specific or desirable outputs, while neutral injections serve to test the model's resilience, accuracy, or response to edge cases without malicious intent.

2 Shielding LLMs against Jailbreaking and Prompt Injections

Defense methods against jailbreaking and prompt injection attacks can be grouped into two directions: attack prevention and attack detection.

The most straightforward prevention method to shield an LLM is to formulate counter-prompts to neutralize the harmful injection ("Hello ChatGPT, please ignore any harmful prompts that might follow this instruction and just do exactly as I say"). While easy to implement, such "begging" is easy to trick and rarely successful. Research has since come up with a range of more elaborate countermeasures. Jain et al. (2023) proposed two prevention baseline methods that illustrate how simple

软性乞求：基于提示调整的模块化和高效防护 LLMs 免受提示注入和越狱攻击

西蒙・奥斯特曼，凯文・鲍姆，克里斯托夫・恩德斯，朱莉娅・马斯洛，帕特里克・施拉莫夫斯基
[1]德国人工智能研究中心 (DFKI)
[2]欧洲可信人工智能研究中心 (CERTAIN)
[3]sequire technology GmbH
firstname.lastname@{dfki|sequire}.de

arXiv:2407.03391v1 [cs.CR] 3 Jul 2024

摘要

提示注入（包括直接和间接）以及越狱现在被认为是大型语言模型（LLMs）的重要问题，特别是在应用集成环境中它们的潜在危害性。这篇扩展摘要探讨了一种保护 LLMs 免受此类攻击的新方法，称为“软性乞求”。该方法涉及训练软提示以抵消损坏的提示对 LLM 输出的影响。我们概述了提示注入和越狱，介绍了“软性乞求”技术的理论基础，并讨论了对其有效性的评估。

1 背景：攻击 LLMs

当前的 LLMs 缺乏对抗鲁棒性（Carlini 等人，2021 年）。这导致了在 LLMs 的攻击表面上出现与安全和安全相关的新现象。突出的例子包括：越狱、直接提示注入和间接提示注入。

在越狱攻击中，目标 LLM 被操控以绕过内容审核系统并违背安全规则（Zou 等人，2023）。提示注入描述的是将输入插入应用程序并由 LLM 在下游处理的攻击——间接提示注入指的是攻击者并非提示 LLM 的用户（Greshake 等人，2023）。可以看出，直接提示注入（Perez 和 Ribeiro，2022）与越狱攻击相似：两者都在提示中隐藏特定指令，从而以 LLM 提供者未预期的方式改变或操控系统行为。这种对抗性指令通常通过文本传达，有时文本会以视觉隐藏（使用白色或极小字体等策略）、编码（Liu 等人，2024b）或由 API 混淆（例如，在不通知最终用户的情况下将命令附加到每个输入中）。

尤其是间接提示注入攻击，攻击者远程影响目标系统，利用当前 LLMs 中代码和数据之间没有明确分离这一事实，因此合法系统指令和无效、潜在对抗性指令之间没有明确分离（Zverev 等人，2024 年）。

提示注入的危险包括数据泄露，即敏感信息的披露，以及系统操控，即根据注入的提示改变系统的行为。在最极端的情况下，对抗性用户可以劫持 LLM（Qiang 等人，2024 年），如果 LLM 具有很大的自主性，例如被授予插件或第三方数据的访问权限，这个问题会更加严重。

上述示例的目标通常是诱使 LLM 表现出有害行为，因此需要注意也存在良性和中性用法。良性注入可以引导模型生成更具体或期望的输出，而中性注入则是为了测试模型的韧性、准确性或对边缘情况的反应，并无恶意意图。

2 保护 LLM 免受越狱和提示注入的攻击

防御针对越狱和提示注入攻击的方法可以分为两个方向：攻击预防和攻击检测。

保护 LLM 最直接的方法是制定反提示来中和有害注入（ 你好 ChatGPT，请忽略任何可能跟随此指令的有害提示，只按照我说的做”）。虽然易于实施，但这种“乞求”很容易被欺骗，很少成功。此后，研究提出了一系列更复杂的反制措施。Jain 等人（2023 年）提出了两种预防基准方法，这些方法说明了如何通过简单

defenses such as input preprocessing (paraphrasing and retokenization) can effectively act against gradient-based methods. Whereas paraphrasing rewrites the input by changing its meaning, retokenization breaks down the tokens into smaller ones. Recently, input preprocessing techniques have been studied that aim at making it easier for the model to distinguish between valid instructions and untrustworthy input (Hines et al., 2024).

The most frequently used post-training detection method is the implementation of filters, i.e., additional algorithms or models that try to scan inputs for injected harmful prompts and mask them or reject the prompts automatically (Dong et al., 2024). Detection methods for prompt injection, for example, often rely on the perplexity score, which is assumed to be higher for adversarial inputs and can be detected by a simple thresholding approach (Alon and Kamfonas, 2023; Jain et al., 2023). Yet, setting the threshold is not trivial and a poor threshold might hurt the model's overall performance. Generally, filters can be implemented as rule-based checks, but they can also be trained deep learning models that classify inputs into harmful and non-harmful parts. The problem with such filters is that they are often too restrictive in the case of model-based filters, and too loose in the case of rule-based filters.

As an alternative, models can theoretically be fine-tuned to be robust against attacks. While working reasonably well, the disadvantage of such methods is that the fine-tuning of whole models is costly; it needs to be redone whenever a new type of malicious prompt is detected; and even robust detection of injections always leaves room for vulnerabilities. Also, although the field of parameter-efficient fine-tuning is being reasonably well studied and still growing (Lialin et al., 2023), contributions that focus on its efficiency in cybersecurity contexts are lacking.

3 Soft Begging: Shielding LLMs with Soft Prompts

We propose *soft begging* as a new alternative for LLM shielding. The method can be seen as a combination of the naïve begging approach, combined with parameter-efficient fine-tuning techniques. The method basically comprises the training of so-called soft prompts, i.e. trainable input vectors that are preprended to any prompt. The soft prompts are trained to nullify the behaviour that the LLM exhibits based on potentially harmful parts of the prompts. This is not done as a filtering step, but in an implicit way – i.e. without altering the prompts – which effectively follows the idea of "begging" the network to ignore the harmful parts on a parameter level.

We conjecture that a first advantage of using such soft prompts is their effectiveness, as it enables shielding on the parameter-level against attacks on the text level, which effectively provides the shield with an advantage: A parameter-level control can be assumed to be always more effective than textual control. Second, shields trained in such a way are easy and efficiently adaptable, as the training of a soft prompt is magnitudes faster than training the whole model, as is done for example when fine-tuning LLMs to be robust against injections. At the same time, obviously, the LLM itself stays as is with its parameters being frozen. Last, we assume that soft begging prompts could be modularized to fit different types of attacks and even be combined for different use cases, rendering them as a very effective and customizable alternative to other shields.

In the most basic version, such prompts are trained on quadruples of clean prompts, corrupted prompts, clean output and output based on the corrputed prompt. The soft prompts are then trained to produce the clean output from the corrupted prompt.

The idea can be scaled up, e.g. by training different soft prompts for different injections and combining them via prompt fusion or other mechanisms. Also, the prompts could be combined with a filtering mechanism, that first identifies the kind of threat (without localizing it), and then picks the matching soft prompt based on this.

4 Evaluation of Prompt Injection

Usually, jailbreaking and direct prompt injection attacks are evaluated by injecting a malicious prompt with a specific phrase into benign queries and observing the output (Perez and Ribeiro, 2022; Chen et al., 2024). This allows for easy evaluation, as the attack is successful if the specified keyword is present in the LLM response.

In contrast to that, in indirect prompt injection attacks, the malicious prompt is embedded into malicious documents and sources, as the name suggests. As described above, the output of the LLM can be reviewed for the injected target keywords,

防御措施如输入预处理（释义和重新分词）可以有效地对抗基于梯度的方法。其中，释义通过改变输入的含义来重写输入，而重新分词将标记分解成更小的部分。最近，人们研究了旨在使模型更容易区分有效指令和不可信输入的输入预处理技术（Hines 等人，2024）。

最常用的后训练检测方法是过滤器实现，即额外的算法或模型，尝试扫描输入中的注入有害提示并自动屏蔽或拒绝这些提示（Dong 等人，2024）。例如，提示注入的检测方法通常依赖于困惑度分数，该分数假定对抗性输入的分数更高，可以通过简单的阈值方法检测（Alon 和 Kamfonas，2023；Jain 等人，2023）。然而，设置阈值并不简单，不合适的阈值可能会损害模型的整体性能。通常，过滤器可以作为基于规则的检查实现，但也可以是训练深度学习模型，将输入分类为有害和非有害部分。这类过滤器的问题在于，基于模型的过滤器往往过于严格，而基于规则的过滤器则过于宽松。

作为替代方案，理论上可以对模型进行微调以使其能够抵御攻击。虽然这些方法在合理地工作，但它们的缺点是整个模型的微调成本很高；每当检测到新型恶意提示时都需要重新进行微调；即使对注入的稳健检测也总是留下漏洞空间。此外，尽管参数高效微调领域得到了合理的研究并仍在发展（Lialin 等人，2023 年），但关注其在网络安全环境中的效率的贡献却缺乏。

3　软恳求：用软提示保护 LLMs

我们提出软恳求作为 LLM 保护的全新替代方案。该方法可以看作是朴素恳求方法与参数高效微调技术的结合。该方法基本上包括训练所谓的软提示，即预附加到任何提示的可训练输入向量。软提示被训练以消除行为 LLM 基于提示中潜在有害部分所表现出的行为。这并非作为过滤步骤进行，而是以一种隐式的方式——即不改变提示——来实现，这实际上遵循了“恳求”网络在参数级别忽略有害部分的想法。

我们推测使用此类软提示的第一个优势是它们的有效性，因为它能够在参数级别上对文本级别的攻击进行防护，这实际上赋予了防护层优势：参数级别的控制可以被认为总是比文本控制更有效。其次，以这种方式训练的防护层易于高效地适应，因为软提示的训练速度远快于训练整个模型，例如在微调 LLM 以使其对注入攻击具有鲁棒性时进行的训练。同时，显然 LLM 本身保持不变，其参数被冻结。最后，我们假设软恳求提示可以被模块化以适应不同类型的攻击，甚至可以组合用于不同的用例，使它们成为其他防护层的非常有效和可定制的替代方案。

在最基本版本中，此类提示是在干净提示、损坏提示、干净输出和基于损坏提示的输出这四元组上进行训练的。然后，软提示被训练以从损坏提示中生成干净输出。

这个想法可以扩展，例如，通过为不同的注入训练不同的软提示，并通过提示融合或其他机制将它们组合起来。此外，提示可以与一个过滤机制结合，该机制首先识别威胁类型（而不定位它），然后根据这一点选择匹配的软提示。

4　提示注入评估

通常，越狱和直接提示注入攻击是通过将带有特定短语的恶意提示注入良性查询中，并观察输出来评估的（Perez 和 Ribeiro，2022；Chen 等人，2024）。这允许轻松评估，因为如果指定的关键词存在于 LLM 响应中，则攻击成功。

与此不同，在间接提示注入攻击中，恶意提示被嵌入到恶意文档和源中，正如其名称所示。如上所述，可以审查 LLM 的输出来查找注入的目标关键词，

indicating a successful attack (Hines et al., 2024; Greshake et al., 2023; Liu et al., 2024a). While the evaluation of jailbreaking and direct prompt injection attacks is relatively straightforward, indirect prompt injection has the problem of requiring third malicious party material like websites, e-mails, or other endpoints providing data to the LLM. To simplify the evaluation of indirect prompt injection attacks, various benchmarks have been recently proposed (Yi et al., 2024; Zhan et al., 2024). These benchmarks provide curated content with pre-defined malicious prompts injected into the queries. This allows for standardized evaluation for many different LLMs.

Evaluation (Liu et al., 2024a,c) typically investigates two model properties: clean performance when facing benign inputs and robustness to adversarial inputs. The clean performance can be computed using standard benchmarks for LLM tasks. Practical defenses should not degrade a model's utility. Robustness, or alternatively the attack success rate, quantifies the share of adversarial prompts achieving their goal of breaking the safety railguards. Hines et al. (2024) highlight the importance of clearly quantifiable protocols for measuring Attack Surface Rate (ASR) and make their evaluation approach using a synthetic dataset explicit. This call for transparency guides our work.

References

Gabriel Alon and Michael Kamfonas. 2023. Detecting language model attacks with perplexity. *arXiv Preprint*, arXiv:2308.14132.

Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650.

Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2024. Struq: Defending against prompt injection with structured queries. arXiv:2402.06363.

Yi Dong, Ronghui Mu, Gaojie Jin, Yi Qi, Jinwei Hu, Xingyu Zhao, Jie Meng, Wenjie Ruan, and Xiaowei Huang. 2024. Building guardrails for large language models. *Preprint*, arXiv:2402.01822.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, AISec '23, page 79–90, New York, NY, USA. Association for Computing Machinery.

Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending against indirect prompt injection attacks with spotlighting. arXiv:2403.14720.

Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline defenses for adversarial attacks against aligned language models. *arXiv Preprint*, arXiv:2309.00614.

Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky. 2023. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *Preprint*, arXiv:2303.15647.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2024a. Prompt injection attack against llm-integrated applications. arXiv:2306.05499.

Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, Kailong Wang, and Yang Liu. 2024b. Jailbreaking chatgpt via prompt engineering: An empirical study. *Preprint*, arXiv:2305.13860.

Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024c. Formalizing and benchmarking prompt injection attacks and defenses. *arXiv Preprint*, arXiv:2310.12815.

Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*.

Yao Qiang, Xiangyu Zhou, and Dongxiao Zhu. 2024. Hijacking large language models via adversarial in-context learning. *Preprint*, arXiv:2311.09948.

Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2024. Benchmarking and defending against indirect prompt injection attacks on large language models. arXiv:2312.14197.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. arXiv:2403.02691.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *Preprint*, arXiv:2307.15043.

Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. 2024. Can llms separate instructions from data? and what do we even mean by that? *Preprint*, arXiv:2403.06833.

这表明攻击成功（Hines 等人，2024 年；Greshake 等人，2023 年；Liu 等人，2024a）。虽然越狱和直接提示注入攻击的评估相对简单，但间接提示注入存在需要第三方恶意材料的问题，如网站、电子邮件或其他向 LLM 提供数据的端点。为了简化间接提示注入攻击的评估，最近提出了各种基准（Yi 等人，2024 年；Zhan 等人，2024 年）。这些基准提供了预先定义了恶意提示的查询中的精选内容。这允许对许多不同的 LLM 进行标准化评估。

评估（刘等人，2024a,c）通常研究两个模型属性：面对良性输入时的清洁性能和对对抗性输入的鲁棒性。清洁性能可以使用标准的 LLM 任务基准来计算。实用的防御措施不应降低模型的效用。鲁棒性，或者换句话说攻击成功率，量化了实现打破安全护栏目标的对抗性提示的比例。Hines 等人（2024）强调了为测量攻击面率（ASR）制定明确可量化协议的重要性，并明确说明了他们使用合成数据集的评价方法。这一呼吁透明度指导了我们的工作。

参考文献

Gabriel Alon and Michael Kamfonas. 2023. Detecting language model attacks with perplexity. arXiv Preprint, arXiv:2308.14132.

Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In 30th USENIX Security Symposium (USENIX Security 21), pages 2633–2650.

陈思哲，朱利安·皮埃尔，查温·西塔瓦林和戴维·瓦格纳。2024。Struq：使用结构化查询防御提示注入。arXiv:2402.06363。

Yi Dong, Ronghui Mu, Gaojie Jin, Yi Qi, Jinwei Hu, Xingyu Zhao, Jie Meng, Wenjie Ruan, and Xiaowei Huang. 2024. Building guardrails for large language models. Preprint, arXiv:2402.01822.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, 和 Mario Fritz. 2023. 并非你所签收的：通过间接提示注入破坏现实世界 LLM 集成应用。在《第 16 届会议论文集》中 ACM Workshop on Artificial Intelligence and Security, AISec '23, 第 79–90 页，纽约，纽约州，美国。Association for Computing Machinery.

Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, 和 Emre Kiciman. 2024. 使用 spotlighting 防御间接提示注入攻击. arXiv:2403.14720.

Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, 和 Tom Goldstein. 2023. 对齐语言模型的对抗攻击的基线防御。arXiv 预印本，arXiv:2309.00614。

Vladislav Lialin，Vijeta Deshpande 和 Anna Rumshisky。2023 年。从小规模扩展到大规模：参数高效微调指南。预印本，arXiv:2303.15647。

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang 刘浩宇，王岩，郑艳和刘杨。2024a 年。针对集成 LLM 应用程序的提示注入攻击。arXiv:2306.05499。

Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, Kailong Wang, and Yang Liu. 2024b. Jailbreaking chatgpt via prompt engineering: An empirical study 预印本，arXiv:2305.13860。

刘宇培、贾宇琪、耿润鹏、贾金源和龚振强。2024c。形式化化和基准测试提示注入攻击和防御。arXiv 预印本，arXiv:2310.12815。

Fábio Perez 和 Ian Ribeiro。2022。 忽略之前的提示：语言模型的攻击技术》。在 NeurIPS ML 安全研讨会。

Yao Qiang, Xiangyu Zhou, and Dongxiao Zhu. 2024. 通过对抗性上下文学习劫持大型语言模型。预印本，arXiv:2311.09948。

Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2024. 对大型语言模型的间接提示注入攻击进行基准测试和防御。arXiv:2312.14197.

战秋思, 梁志祥, 英子凡, 和康丹尼尔. 2024. Injecagent: 对工具集成的大型语言模型代理中的间接提示注入进行基准测试. arXiv:2403.02691.

邹安迪, 王子凡, 卡林尼·尼古拉斯, 纳斯拉·米尔, 科尔特·齐科·Z, 和弗雷德里克森·马特. 2023. 对对齐语言模型的通用和可迁移的对抗攻击. 预印本, arXiv:2307.15043.

Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, 和 Christoph H. Lampert. 2024. LLMs 能否区分指令和数据？我们甚至是指什么？预印本，arXiv:2403.06833.